

Modul 11 Basis Data
PRAKTIKUM BASIS DATA

NoSQL Key Value Database



DISUSUN OLEH:

Ryvanda

103122400027

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026

1. Perbedaan Mendasar RDBMS vs Apache Cassandra & Alasan Disebut "Schema-less"

Biar gampang dibayangin, RDBMS (SQL) itu kayak perumahan berpagar yang tipe rumahnya wajib seragam (semua kamar kudu ada kasur, lemari, dan meja). Sedangkan Apache Cassandra (NoSQL) itu kayak lahan bebas; kamar si A cuma diisi kasur doang boleh, kamar si B mau ditambahin sofa, kulkas, sama TV juga bebas.

Berikut adalah poin-poin perbedaan mendasarnya:

- Struktur Data: SQL itu kaku karena berbasis tabel yang baris dan kolomnya wajib kembar. Kalau Cassandra pake konsep *Column-Family*, di mana tiap baris data dibebaskan punya jumlah dan jenis kolom yang beda-beda.
- Aturan Skema (*Schema*): Di SQL, lu wajib ngunci desain tabel di awal (*schema-on-write*), kalau mau diubah di tengah jalan bakal ribet. Nah, Cassandra sifatnya dinamis atau *schema-less*, lu bisa nambahin kolom baru kapan aja tanpa ngerusak tatanan tabel yang udah ada.
- Hubungan Data (Relasi): SQL itu rajanya relasi karena dukung perintah JOIN pake kunci tamu (*Foreign Key*). Kalau Cassandra gak mainan JOIN langsung; trik mereka adalah mendenormalisasi data (satu data besar ditaruh di satu tempat) biar proses bacanya secepat kilat.
- Cara Upgrade (*Scalability*): Kalau database SQL lu kepenuhan, lu kudu upgrade komputer/server lu ke spesifikasi yang lebih mahal (Skalabilitas Vertikal). Kalau Cassandra tinggal nambahin PC/node biasa aja buat bareng-bareng gotong royong nampung datanya (Skalabilitas Horizontal).

Kenapa Cassandra Disebut "Schema-less"?

Sederhananya, karena Cassandra gak matokin aturan kalau tiap baris data harus punya struktur kolom yang kembar identik. Kolom bisa lu lepas-pasang secara dinamis tanpa perlu ngelakuin ritual migrasi database yang bikin pusing. Tapi secara teknis, karena dia masih punya definisi tabel induk di awal, orang-orang sering menyebutnya sebagai Semi Schema-less.

2. Urusan Tambah Kolom Mendadak: Cassandra vs SQL

Bayangin lu punya aplikasi yang udah jalan, tiba-tiba lu butuh nambahin satu kolom baru di tengah jalan. Ini cara kedua kubu mengatasinya:

A. Gaya SQL (RDBMS)

Lu bakal ngetik perintah andalan:

```
ALTER TABLE nama_tabel ADD COLUMN kolom_baru VARCHAR(255);
```

Resikonya: Perintah ini berpotensi mengunci tabel transaksi lu (*lock table*). Kalau datanya udah jutaan, aplikasi lu bisa ngadat atau *down* sejenak, dan lu butuh rencana matang biar gak diamuk user.

B. Gaya Cassandra

Di Cassandra, lu tinggal masukin aja data baru beserta kolom barunya.

- Data lama lu bakal tetep aman dan valid meskipun mereka gak punya kolom baru tersebut.
- Gak ada drama database kekunci atau migrasi skema yang bikin senam jantung.
- Contohnya: Baris data si A cuma punya kolom nama dan kategori, tapi baris data si B bisa langsung punya kolom nama, kategori, plus tambahan kolom harga di dalam satu tabel yang sama.

Kenapa Cassandra Lebih Juara buat Aplikasi Web Dinamis?

Web zaman sekarang (kayak e-commerce atau media sosial) itu perkembangannya cepet banget, makanya Cassandra laku banget di sana karena beberapa alasan:

1. Update Tanpa Matol (*No Downtime*): Developer bisa bebas nambahin fitur atau atribut baru di aplikasi tanpa perlu mematikan server database.
2. Siap Menampung Data "Belang-Belang": Sangat fleksibel buat aplikasi modern yang isi datanya gak pernah seragam, contohnya kayak variasi isi profil user, data log aktivitas, atau kiriman sensor IoT.
3. Mangkas Waktu Kerja (*Agile Friendly*): Karena gak butuh migrasi skema yang ribet, tim developer bisa kerja sat-set pakai metode *Agile* buat rilis fitur baru tiap minggu tanpa terhambat urusan database.